

Conceptos intermedios

19. Mostrar contenido

- **display** : Propiedad fundamental que determina el modelo de caja y el contexto de formato que un elemento genera en el motor de renderizado del navegador, su función técnica es dictar cómo se comporta un elemento respecto a sus vecinos (si salta de línea o se mantiene al lado) y cómo se comportan sus hijos (si se organizan como texto, como una rejilla o como cajas flexibles).

Se utiliza como el interruptor principal para cambiar la arquitectura de diseño de cualquier componente; permite transformar un elemento estructural simple en un contenedor de alta tecnología como Flexbox o Grid, o incluso eliminarlo por completo del flujo visual, siendo la directiva con mayor impacto en la geometría y el comportamiento espacial de toda la interfaz web.

— **block** : Transforma al elemento en una caja de bloque de ancho completo, lo que significa que ocupa todo el espacio horizontal disponible de su contenedor padre por defecto, técnicamente un elemento en bloque siempre comienza en una nueva línea y empuja a cualquier elemento siguiente hacia abajo en el eje vertical.

Se utiliza para construir la estructura principal y el esqueleto del sitio web; permite el control total de todas las dimensiones del modelo de caja (ancho, alto, márgenes y rellenos), siendo la base para definir secciones, párrafos, encabezados y contenedores genéricos que segmentan el contenido.

— **inline** : Hace que el elemento se comporte como parte del flujo de texto, ocupando únicamente el ancho mínimo necesario para contener su información, el motor de renderizado permite que múltiples elementos de línea se sitúen uno al lado del otro en el mismo renglón hasta que se agote el espacio.

Se utiliza para estilizar fragmentos específicos de contenido sin romper la continuidad de la lectura, técnicamente los elementos de línea ignoran las propiedades de width, height y los márgenes verticales, lo que los hace ideales para enlaces, negritas o etiquetas de énfasis dentro de un párrafo.

— **inline-block** : Valor híbrido que combina la disposición horizontal de los elementos de línea con las capacidades de dimensionamiento de los elementos de bloque, permitiendo que los elementos se sitúen uno al lado del otro en la misma línea, pero conservando la capacidad técnica de aceptar propiedades de ancho, alto, márgenes y rellenos verticales.

Se utiliza estratégicamente para crear componentes de navegación y galerías donde se requiere que los elementos tengan un tamaño específico y consistente, pero que fluyan lateralmente de manera armoniosa sin necesidad de recurrir a sistemas de posicionamiento más complejos.

— **none** : Elimina completamente al elemento del árbol de renderizado, lo que significa que el navegador actúa como si el nodo no existiera en absoluto.

Se emplea habitualmente en interacciones dinámicas y menús desplegados,

permitiendo ocultar o mostrar secciones enteras de la interfaz mediante JavaScript o estados de CSS sin dejar rastros visuales ni interferir en la geometría de la página.

— **list-item** : Hace que el elemento se comporte técnicamente como un bloque, pero con la característica adicional de generar una caja de marcador (normalmente un punto o un número) en su margen izquierdo, el motor de CSS habilita el uso de propiedades específicas de listas, como `list-style-type`, para controlar la apariencia de dicho marcador. Se utiliza para definir la estructura de listados y viñetas, aunque es el comportamiento por defecto de la etiqueta `<li>`, puede aplicarse a cualquier otro elemento para convertirlo visualmente en un ítem de lista sin alterar la semántica original del HTML.

20. posicionamientos

- **position** : Define el algoritmo de posicionamiento que el motor de renderizado debe aplicar a un elemento, determinando su ubicación final en el plano bidimensional de la pantalla.

Su función técnica es establecer un sistema de coordenadas de referencia que permite desplazar el elemento mediante las propiedades auxiliares `top`, `right`, `bottom` y `left`, se utiliza para romper o alterar el flujo normal del documento, permitiendo desde ajustes sutiles de alineación hasta la creación de capas superpuestas (capas Z) o elementos que permanecen estáticos mientras el usuario se desplaza por la página, siendo la herramienta fundamental para construir interfaces complejas como menús pegajosos, ventanas modales y decoraciones flotantes.

— **static** : Valor por defecto de todos los elementos HTML y sitúa al componente según el flujo normal del documento, bajo este esquema el elemento se posiciona estrictamente donde le corresponde según su orden en el código, y las propiedades de desplazamiento (`top`, `right`, `bottom`, `left`) así como el `z-index` no tienen ningún efecto técnico sobre él. Se utiliza para mantener la estructura natural y predecible de la página, donde cada bloque empuja al siguiente de forma secuencial, garantizando que el diseño respete las reglas básicas de apilamiento y alineación sin intervenciones manuales de coordenadas.

— **relative** : Posiciona al elemento según el flujo normal, pero permite desplazarlo de su ubicación original mediante las propiedades de dirección sin que el espacio que ocupaba inicialmente sea liberado, técnicamente el navegador reserva el hueco original del elemento y mueve solo su representación visual, además convierte al elemento en un punto de referencia para cualquier hijo que utilice posicionamiento absoluto. Se utiliza para realizar ajustes finos de alineación (como mover un icono unos píxeles) o para servir de contenedor "ancla" en componentes complejos donde los sub-elementos deben posicionarse respecto a este padre.

— **absolute** : Extrae al elemento completamente del flujo normal del documento, lo que significa que ya no ocupa espacio físico para sus hermanos y estos se

reorganizan como si el elemento no existiera, el componente se posiciona respecto a su ancestro posicionado más cercano (cualquiera que no sea static), si no encuentra ninguno, se orienta respecto al cuerpo del documento (body).

Se emplea estratégicamente para crear elementos superpuestos, como etiquetas de "oferta" sobre una imagen, menús desplegables o iconos decorativos que deben flotar en un punto exacto de un contenedor sin desplazar el texto o el contenido circundante.

— **fixed** : Retira al elemento del flujo del documento, pero con la diferencia crítica de que lo posiciona respecto a la ventana gráfica (viewport) del navegador, esto implica que el elemento permanecerá estático en una coordenada exacta de la pantalla incluso cuando el usuario haga scroll por la página.

Se utiliza de forma estándar para construir barras de navegación persistentes, botones de "ir arriba", chats de soporte técnico o avisos legales que deben estar siempre visibles para el usuario, garantizando que el componente mantenga su jerarquía visual fija sobre todo el contenido desplazable.

— **sticky** : Valor híbrido que se comporta como relative hasta que el elemento alcanza un umbral de desplazamiento específico definido (por ejemplo, top: 0), momento en el cual se queda "pegado" y actúa como fixed dentro de los límites de su contenedor padre. Una vez que el contenedor padre sale de la vista, el elemento se desplaza con él.

Se utiliza para crear encabezados de secciones o tablas que deben permanecer visibles mientras el usuario lee el contenido de ese bloque específico, ofreciendo una experiencia de navegación contextual muy fluida que mejora la usabilidad en documentos extensos o listas de datos.

- **top** : Establece la distancia de desplazamiento vertical de un elemento posicionado respecto al borde superior de su bloque de contención, en elementos con posicionamiento absolut define la ubicación exacta del borde superior de la caja dentro de su ancestro de referencia, en posicionamiento relativo desplaza la caja visual hacia abajo respecto a su ubicación original sin afectar el espacio que ocupa en el flujo del documento.
Se utiliza para el control preciso de la coordenada Y en la interfaz, permitiendo situar componentes como encabezados pegajosos, notificaciones flotantes o elementos decorativos en una altura específica de la pantalla o del contenedor.
- **right** : Define la distancia horizontal entre el borde derecho de un elemento posicionado y el borde derecho de su bloque de referencia, el motor de renderizado utiliza este valor para empujar el elemento hacia la izquierda desde el límite lateral derecho.
Se utiliza estratégicamente en la alineación lateral inversa, permitiendo que elementos como botones de "cerrar" en ventanas modales, menús laterales (drawers) o insignias de notificación se anclen de forma consistente al extremo derecho de sus contenedores, independientemente de cuánto cambie el ancho total del elemento o de la resolución de la pantalla del usuario.

- **bottom** : Determina el desplazamiento vertical de un elemento posicionado respecto al borde inferior de su bloque de contención, funciona estableciendo una distancia de separación que empuja el elemento hacia arriba desde la base de su referencia. Se emplea habitualmente en el diseño de barras de herramientas inferiores, botones de "ir arriba", pies de página persistentes o leyendas de imágenes, garantizando que el componente mantenga una relación constante con la parte baja del contenedor o del viewport, facilitando la creación de interfaces que aprovechan el espacio inferior de los dispositivos móviles.
- **left** : Establece la distancia de desplazamiento horizontal de un elemento posicionado respecto al borde izquierdo de su bloque de referencia, es la propiedad fundamental para definir el inicio de la coordenada X en sistemas de coordenadas cartesianas dentro de la web.
Se utiliza para desplazar elementos hacia la derecha de su posición original o para anclarlos en un punto exacto desde el origen lateral izquierdo; es esencial en la creación de barras laterales de navegación, posicionamiento de tooltips o cualquier componente que requiera una alineación rígida y predecible desde el margen izquierdo del diseño.
- **z-index** : Gestiona el orden de apilamiento de los elementos en el eje de profundidad (eje Z), determinando qué componentes aparecen delante o detrás de otros cuando se produce un solapamiento visual, el navegador procesa este valor numérico para crear un contexto de apilamiento (stacking context), donde los valores enteros más altos se renderizan sobre los valores más bajos.
Se utiliza de forma crítica para mantener la jerarquía de capas en aplicaciones web complejas, asegurando que elementos de alta prioridad como modales, menús desplegables y alertas de sistema no queden ocultos tras el contenido principal o elementos decorativos.

21. Visibilidad

- **visibility** : Propiedad de renderizado que determina si la caja de un elemento es perceptible visualmente o si debe permanecer oculta, afectando su visibilidad sin alterar su ocupación en el layout, a diferencia de otras directivas de ocultación, cuando se aplica el valor hidden, el motor de CSS hace que el elemento sea invisible y que no responda a eventos de puntero (clics o hovers), pero mantiene intacto el espacio físico que el elemento ocupa en el árbol del DOM.
Se utiliza de forma estratégica para gestionar la estabilidad del diseño en interfaces dinámicas, por ejemplo para ocultar iconos o textos de error que deben aparecer y desaparecer sin que el resto de los componentes vecinos se reorganicen o "salten" al cambiar el estado, preservando la geometría y las dimensiones originales del contenedor padre independientemente del estado visual del hijo.

|
| — **visible** : Valor por defecto de la propiedad visibility que indica al motor de renderizado que el elemento debe ser dibujado de forma normal en la interfaz, bajo este estado, el elemento es totalmente perceptible, ocupa su espacio físico en el layout y responde a todas las interacciones del usuario (como clics, selección de texto o eventos de puntero).
| Se utiliza para restaurar la presencia visual de componentes que han sido

ocultos previamente o para asegurar que elementos críticos de la navegación permanezcan siempre en el plano de interacción, garantizando que el flujo de renderizado procese tanto la geometría como la estética del nodo sin restricciones de transparencia o interactividad.

hidden : Instruye al navegador para que deje de renderizar visualmente el elemento, haciéndolo completamente invisible y desactivando su capacidad de respuesta a eventos de interacción, la característica técnica más importante de hidden es que el elemento continúa ocupando su espacio físico en el árbol del documento, es decir el layout no se altera y los elementos vecinos no se mueven para llenar el vacío, dejando un hueco exacto de las mismas dimensiones que el elemento original.
Se utiliza estratégicamente en interfaces dinámicas y animaciones para ocultar información sensible o decorativa sin provocar saltos visuales o reajustes geométricos bruscos en el resto de la página, manteniendo la estabilidad estructural del diseño.

22. Overflow

- **¿Qué es?** : Controla el comportamiento del motor de renderizado cuando el contenido de un elemento excede las dimensiones físicas de su caja (su ancho o su alto definidos), su función técnica es determinar si el contenido sobrante debe ser recortado, si debe mostrarse fuera de los límites del contenedor o si el navegador debe proporcionar mecanismos de desplazamiento para acceder a la información oculta.

Se utiliza de forma crítica en el diseño de interfaces con contenido dinámico, permitiendo mantener la integridad estética del layout al evitar que textos largos o imágenes de gran tamaño rompan la estructura de la página, proporcionando un control granular sobre cómo el usuario interactúa con los datos que superan el espacio visual asignado al componente.

overflow-x : Controla de forma específica el comportamiento del motor de renderizado cuando el contenido de un elemento excede el ancho (eje horizontal) de su caja, su función técnica es determinar si el excedente lateral debe ser recortado, si debe mostrarse fuera de los límites o si el navegador debe habilitar una barra de desplazamiento horizontal.
Se utiliza estratégicamente en el diseño de componentes anchos, como tablas de datos complejas, carruseles de imágenes o bloques de código extenso en dispositivos móviles, permitiendo que el usuario explore el contenido deslizándose lateralmente sin romper la estructura vertical ni la alineación general del resto de la página.

overflow-y : Define cómo debe actuar el navegador cuando el contenido interno de un elemento sobrepasa la altura (eje vertical) asignada a su contenedor, técnicamente, permite restringir el crecimiento de un elemento en el DOM y gestionar el excedente mediante el recorte o la activación de una barra de desplazamiento vertical.
Se emplea de forma estándar en la creación de áreas de lectura controladas, como secciones de comentarios, modales con mucho texto o menús

desplegables largos, garantizando que el diseño de la interfaz permanezca compacto y que el usuario pueda navegar por el contenido extenso sin desplazar forzosamente el resto de los elementos del layout.

23. Flexbox

- **display: flex** : Declaración que transforma a un elemento en un contenedor flexible, alterando radicalmente el contexto de formato de todos sus hijos directos (que pasan a ser "flex items"), técnicamente el motor de renderizado deja de seguir las reglas tradicionales de bloque o línea para estos elementos y comienza a distribuirlos a lo largo de un eje principal (main axis) y un eje cruzado (cross axis). Se utiliza como la herramienta fundamental para el diseño de layouts dinámicos y alineaciones complejas, permitiendo que los elementos se expandan para llenar el espacio sobrante, se encojan para evitar desbordamientos o se alineen con precisión matemática (tanto horizontal como verticalmente) sin necesidad de cálculos manuales de márgenes o el uso de flotados, garantizando una interfaz fluida que se adapta a cualquier tamaño de pantalla.

- **Contenedor flex**

— **flex-direction** : Establece el eje principal sobre el cual se desplazarán los elementos hijos dentro del contenedor flexible, determinando la dirección del flujo de contenido, sus valores permiten organizar los elementos en filas horizontales (row), columnas verticales (column) o sus respectivas variantes invertidas (reverse), técnicamente define si el "inicio" y el "fin" del contenedor se calculan en el eje X o en el eje Y. Se utiliza de forma estratégica para conmutar la estructura del layout entre dispositivos móviles y de escritorio, permitiendo que un menú horizontal en PC se transforme en una lista vertical en teléfonos con una sola regla de estilo.

— **flex-wrap** : Determina si los elementos hijos deben mantenerse en una sola línea o si el motor de renderizado tiene permiso para fragmentarlos en múltiples líneas cuando el espacio del contenedor se agota, por defecto Flexbox intenta ajustar todos los elementos en una fila única, lo que puede provocar desbordamientos o encogimientos excesivos. Se utiliza en la creación de galerías de tarjetas o rejillas flexibles, donde se desea que los elementos fluyan de forma natural hacia el siguiente renglón a medida que la ventana del navegador se reduce, garantizando que el contenido permanezca legible y dentro de los límites visuales del componente.

— **justify-content** : Define cómo se distribuye el espacio sobrante entre los elementos hijos a lo largo del eje principal (el eje definido por flex-direction), el navegador utiliza este valor para alinear el grupo de elementos al inicio, al centro, al final, o para repartir el aire de forma equitativa mediante espacios entre ellos (space-between, space-around, space-evenly). Se emplea de forma estándar para el centrado horizontal y la separación de componentes, permitiendo por ejemplo que el logotipo de una web se mantenga a la izquierda y el menú a la derecha de forma automática y proporcional al ancho disponible.

— **align-items** : Controla la alineación de los elementos hijos a lo largo del eje cruzado (el eje perpendicular al principal), su función técnica es determinar la posición vertical de los elementos cuando están en una fila, o la posición horizontal cuando están en una columna, permitiendo que todos compartan una altura común o se alineen respecto a su línea base o centro.
Es la herramienta definitiva para lograr el centrado vertical perfecto, una tarea históricamente compleja en CSS, asegurando que elementos de diferentes alturas se vean armoniosos y equilibrados dentro de un mismo contenedor.

— **align-content** : Actúa sobre las líneas de contenido completas cuando el contenedor tiene múltiples filas o columnas (debido al uso de flex-wrap), el motor de renderizado utiliza este valor para distribuir el espacio vertical sobrante entre las diferentes líneas de elementos.
Se utiliza exclusivamente en layouts multilínea complejos para evitar que queden huecos irregulares entre las filas de una galería o rejilla, permitiendo agrupar todas las líneas al centro o separarlas uniformemente para mantener una estética de cuadrícula cohesiva.

- **Elementos flex**

— **order** : Altera el orden de visualización de un elemento hijo sin modificar su posición real en el código fuente HTML, el navegador utiliza un sistema de valores enteros (por defecto 0) para reordenar los elementos de menor a mayor valor, los elementos con números negativos aparecen primero y los positivos después.
Se utiliza estratégicamente en el diseño responsivo para priorizar contenido visualmente en dispositivos móviles (como subir una barra lateral a la parte superior) sin romper la estructura semántica original del documento, permitiendo que la lectura para motores de búsqueda y lectores de pantalla permanezca lógica mientras la estética se adapta.

— **flex-grow** : Factor de expansión que determina cuánto espacio sobrante del contenedor principal debe asignarse a un elemento hijo específico, técnicamente, funciona como una proporción de crecimiento: si un elemento tiene flex-grow: 2 y sus hermanos tienen 1, el primero recibirá el doble de espacio libre que los demás.
Se emplea habitualmente en la creación de layouts elásticos, permitiendo que secciones de contenido principal se expandan automáticamente para llenar el ancho total de la pantalla mientras que las barras laterales mantienen un tamaño más contenido, garantizando que el diseño aproveche cada píxel disponible de forma proporcional.

— **flex-shrink** : Define la capacidad de un elemento para encogerse cuando el tamaño total de los hijos supera el espacio disponible en el contenedor padre, el valor asignado actúa como un factor de resistencia: a mayor número, más rápido se reducirá el elemento para evitar el desbordamiento.
Se utiliza para prevenir la ruptura del layout en pantallas pequeñas, por

ejemplo permite que elementos secundarios o decorativos cedan su espacio y se compriman antes que los elementos críticos de navegación o texto, asegurando que la interfaz se mantenga dentro de los límites visuales del dispositivo sin generar barras de desplazamiento accidentales.

— **flex-basis** : Define el tamaño inicial predeterminado de un elemento antes de que se reparta el espacio sobrante mediante las reglas de crecimiento o encogimiento, a diferencia de width o height, esta propiedad establece la base de cálculo en el eje principal (ya sea horizontal o vertical).

Se utiliza para establecer un punto de partida dimensional rígido para los componentes, el motor de renderizado toma este valor como la medida ideal del elemento, permitiendo que el diseño sea predecible antes de que entren en juego las dinámicas de flexibilidad de Flexbox, siendo esencial para definir anchos de columnas base en sistemas de rejilla.

24. Grid

- **display: grid** : Declaración que activa el modelo de diseño bidimensional más potente de CSS, transformando al elemento en un contenedor de rejilla (grid container), a diferencia de Flexbox, que se orienta en una sola dirección, Grid permite al motor de renderizado organizar los elementos hijos simultáneamente en filas y columnas.

Su función técnica es establecer un sistema de coordenadas rígido pero flexible donde se definen áreas, huecos y alineaciones con una precisión estructural absoluta, además se utiliza como la herramienta definitiva para la arquitectura de layouts complejos, permitiendo crear interfaces de páginas completas, paneles de control o galerías asimétricas sin depender de márgenes negativos o floats, garantizando que la estructura visual se mantenga perfectamente alineada tanto horizontal como verticalmente en cualquier resolución.

— **grid-template-columns** : Define la estructura horizontal de la rejilla al especificar el número y el ancho de las columnas dentro del contenedor, el motor de renderizado utiliza una lista de valores (como px, %, fr o palabras clave) para segmentar el espacio disponible.

Se utiliza para establecer el esqueleto lateral del layout; cada valor declarado crea una nueva guía vertical, permitiendo diseñar desde rejillas simples de dos columnas hasta estructuras asimétricas complejas de forma explícita y predecible, garantizando que todos los elementos hijos se alineen perfectamente en el eje X.

— **grid-template-rows** : Define la estructura vertical de la rejilla al determinar la cantidad y la altura de las filas del contenedor, al igual que con las columnas, el navegador procesa los valores asignados para reservar espacio en el eje Y. Se emplea para controlar la jerarquía de altura en el diseño, por ejemplo permite fijar una altura exacta para el encabezado (header) y el pie de página (footer) mientras se deja que el contenido central crezca de forma automática, asegurando que la página mantenga una organización vertical coherente independientemente del volumen de información.

— **repeat()** : Función de notación abreviada que permite definir múltiples columnas o filas que comparten el mismo patrón de tamaño sin necesidad de escribir cada valor individualmente, su sintaxis técnica, `repeat(veces, tamaño)`, instruye al navegador para que duplique la estructura indicada el número de veces especificado.

Se utiliza para optimizar la legibilidad del código en layouts repetitivos, por ejemplo en lugar de escribir cinco veces una medida, se usa esta función para crear rejillas uniformes de forma compacta, facilitando el mantenimiento y la escalabilidad del diseño.

— **minmax()** : Define un rango de tamaño dinámico para una columna o fila, estableciendo un límite mínimo y un límite máximo de crecimiento, el motor de CSS garantiza que el espacio nunca sea menor al primer parámetro ni mayor al segundo, adaptándose fluidamente entre ambos valores según el espacio disponible en el contenedor.

Se utiliza de forma estratégica para el diseño responsivo sin media queries, permitiendo que las columnas se compriman hasta un punto seguro de legibilidad y se expandan hasta un tope estético, evitando que el contenido colapse o se desproporcione.

— **fr** : Representa una fracción del espacio libre disponible dentro del contenedor de rejilla después de haber descontado los tamaños fijos, es una unidad de medida elástica diseñada exclusivamente para CSS Grid.

Se emplea para crear diseños proporcionales y fluidos, por ejemplo si se definen dos columnas como `1fr` y `2fr`, la segunda siempre ocupará el doble del espacio sobrante que la primera, eliminando la necesidad de realizar cálculos porcentuales complejos y asegurando que la rejilla siempre llene el contenedor de forma armónica.

— **grid-column** : Propiedad abreviada determina la ubicación y extensión horizontal de un elemento hijo dentro de la rejilla, indicando en qué línea de columna debe empezar y en cuál debe terminar, técnicamente utiliza los números de las líneas de la rejilla o la palabra clave `span` para ocupar múltiples celdas.

Se utiliza para que elementos específicos resalten en el layout, por ejemplo permite que un anuncio o un encabezado de sección se expanda a lo largo de varias columnas mientras el resto de los elementos mantienen su tamaño estándar de celda.

— **grid-row** : Define la posición y el alcance vertical de un elemento dentro de la estructura de filas de la rejilla, permitiendo especificar los puntos de inicio y fin del nodo mediante los índices de las líneas de rejilla.

Se utiliza para crear diseños asimétricos y dinámicos, permite que un elemento lateral (como una barra de navegación o un banner) se estire verticalmente para cubrir varias filas, permitiendo una organización visual del contenido que rompe con la linealidad tradicional de las cajas apiladas.

— **gap** : Establece el espacio de separación (canaletas o gutters) entre las filas y

las columnas de la rejilla, sin añadir margen en los bordes exteriores del contenedor, el navegador calcula este hueco de forma automática sin afectar el tamaño de las celdas definidas.

Se utiliza para dar respiración y claridad visual al diseño, es la herramienta estándar para separar tarjetas o bloques de contenido, evitando que los elementos se toquen entre sí y mejorando la legibilidad del conjunto sin necesidad de manipular márgenes individuales en cada elemento hijo.

— **place-items** : Propiedad abreviada que permite controlar simultáneamente la alineación horizontal y vertical de todos los elementos hijos dentro de sus celdas individuales. Combina las funciones de align-items (por ejemplo vertical) y justify-items (eje horizontal).

Se utiliza para lograr el centrado absoluto de contenido de forma extremadamente sencilla, al aplicar place-items: center, el navegador sitúa automáticamente cada elemento justo en el centro de su espacio asignado en la rejilla, garantizando una simetría perfecta en componentes como galerías o paneles de control.

— **place-content** : Gestiona la alineación y distribución del total de la rejilla (el bloque completo de filas y columnas) dentro de su contenedor padre, siempre que la rejilla sea más pequeña que dicho contenedor, combinando las acciones de align-content y justify-content.

Se emplea para posicionar el sistema de rejilla completo en el espacio disponible, permite por ejemplo centrar una cuadrícula de imágenes de tamaño fijo en medio de una pantalla ancha o repartir las filas uniformemente para ocupar toda la altura del viewport, controlando el aire sobrante del layout global.

— **auto-fill** : Palabra clave utilizada dentro de la función repeat() que indica al navegador que cree tantas columnas como quepan en el contenedor basándose en el tamaño mínimo proporcionado, incluso si algunas columnas quedan vacías.

Su función técnica es maximizar el número de celdas por fila y se utiliza en el diseño de rejillas ultra-responsivas, permitiendo por ejemplo que un listado de productos se reorganice automáticamente de 4 a 1 columnas según el ancho de la pantalla sin necesidad de código adicional, aprovechando cada espacio lateral disponible para insertar nuevas celdas.

— **auto-fit** : Palabra clave que se utiliza dentro de la función repeat() para crear un sistema de columnas que se ajusta de forma inteligente al ancho del contenedor, con la característica técnica de colapsar las columnas vacías una vez que todos los elementos disponibles han sido posicionados, a diferencia de otros métodos de llenado, el motor de renderizado estira las columnas que sí contienen elementos para que ocupen todo el espacio sobrante del eje horizontal, eliminando cualquier hueco al final de la fila.

Se utiliza para diseñar rejillas perfectamente fluidas y responsivas sin necesidad de media queries, garantizando que, si hay pocos elementos (como en un resultado de búsqueda con solo dos productos), estos se expandan

proporcionalmente para cubrir todo el ancho de la pantalla, manteniendo una estética de bloque sólido y equilibrado.

25. Responsive design

- **@media** : Regla de arroba (at-rule) fundamental que permite aplicar bloques de estilos CSS de forma condicional basándose en las características del dispositivo o del entorno del navegador, técnicamente el motor de renderizado evalúa una "consulta de medios" (media query) que comprueba factores como el ancho de la ventana, la resolución de pantalla o el tipo de medio (pantalla vs. impresora). Se utiliza como el pilar del diseño responsivo, permitiendo que una misma página HTML transforme radicalmente su layout, tipografía y colores para ofrecer una experiencia de usuario optimizada tanto en teléfonos móviles como en monitores de escritorio, garantizando la adaptabilidad total del sitio.

— **orientation** : Característica de medios (media feature) que permite al navegador detectar la relación de aspecto actual de la ventana gráfica (viewport) para aplicar estilos específicos según cómo se esté sosteniendo el dispositivo. Técnicamente, determina si el ancho de la pantalla es mayor que su alto o viceversa.
Se utiliza estratégicamente en el desarrollo de aplicaciones móviles y sitios web de lectura, permitiendo modificar la disposición de elementos críticos (como menús laterales que pasan a ser inferiores) cuando el usuario gira su tableta o teléfono, asegurando que la interfaz aproveche siempre la dimensión más amplia disponible de forma inteligente.

— **portrait** : Valor específico de la consulta de orientación que se activa cuando la altura de la ventana gráfica es mayor o igual que su anchura, representando el estado de "verticalidad" del dispositivo, típico del uso natural de los teléfonos inteligentes.
Se emplea para aplicar diseños de columna única y optimizaciones de lectura vertical, permitiendo ajustar el tamaño de las imágenes para que no se corten lateralmente y reorganizar los elementos de navegación para que sean fácilmente alcanzables con el pulgar, garantizando una usabilidad ergonómica cuando el espacio horizontal es el más restrictivo.

— **landscape** : Valor de orientación que se activa cuando la anchura de la ventana gráfica es mayor que su altura, representando el estado "apaisado" o de visualización panorámica, el motor de CSS utiliza esta condición para identificar monitores de escritorio, tabletas en posición horizontal o teléfonos girados.
Se utiliza para maximizar el uso del espacio lateral; permite desplegar rejillas multicolumna, barras de herramientas laterales y fondos de pantalla extendidos que en el modo vertical resultarían excesivamente estrechos, asegurando que el contenido visual aproveche la amplitud horizontal para mostrar más información de forma simultánea.

- **Media queries avanzadas**

- **prefers-reduced-motion** : Consulta de medios permite detectar si el usuario ha activado una configuración en su sistema operativo para minimizar la cantidad de movimiento no esencial en pantalla, técnicamente el navegador consulta las preferencias de accesibilidad del sistema para informar al CSS si debe reducir, simplificar o eliminar animaciones y transiciones automáticas. Se utiliza para proteger a usuarios con trastornos vestibulares, epilepsia fotosensible o cinetosis (mareo por movimiento), permitiendo que el desarrollador desactive efectos de desplazamiento paralaje, rebotes de menús o zooms agresivos, garantizando una navegación segura y confortable sin sacrificar la funcionalidad.
- **prefers-color-scheme** : Directiva técnica que identifica si el usuario prefiere un tema de interfaz claro (light) o oscuro (dark) basándose en la configuración global de su sistema operativo o agente de usuario, el motor de CSS utiliza esta condición para conmutar paletas de colores de forma automática mediante variables.
Se utiliza como el estándar moderno para la implementación del Modo Oscuro, permite que la página web se mimetice con el entorno visual del dispositivo del usuario (como macOS o Android en modo noche), reduciendo la fatiga visual en entornos de poca luz y mejorando el ahorro de batería en pantallas de tecnología OLED.
- **prefers-contrast** : Detecta si el usuario solicita que el contenido se muestre con un nivel de contraste mayor o menor entre los elementos de primer plano y el fondo, el navegador evalúa las preferencias de accesibilidad del sistema para indicar al diseño si debe forzar bordes más marcados, colores más saturados o eliminar transparencias.
Se emplea de forma crítica para mejorar la legibilidad para personas con baja visión o daltónicas, permitiendo que la hoja de estilos aplique una jerarquía visual mucho más agresiva y nítida que asegure que el texto y los controles de navegación sean inequívocamente distinguibles.
- **hover** : Detecta si el dispositivo principal de entrada del usuario tiene la capacidad física de situar un cursor sobre los elementos sin activarlos (como un mouse o un trackpad), técnicamente permite al CSS diferenciar entre una computadora y una pantalla táctil.
Se utiliza para optimizar la interactividad de la interfaz, permite ocultar efectos visuales complejos que solo funcionan con puntero o evitar que menús desplegables se queden "atrapados" en un estado abierto en dispositivos móviles donde no existe el concepto de "quitar el mouse" de encima.
- **pointer** : Evalúa la precisión del dispositivo de entrada principal del usuario, clasificándolo en valores como fine (precisión alta, como un mouse o stylus) o coarse (precisión baja, como un dedo en una pantalla táctil), el motor de renderizado informa sobre el hardware físico para que el diseño se adapte dinámicamente.
Se utiliza para ajustar la ergonomía de los controles, por ejemplo permite

aumentar automáticamente el tamaño de los botones, el espaciado de los enlaces o el grosor de las barras de desplazamiento cuando se detecta un puntero grueso (coarse), garantizando que la interfaz sea fácil de operar sin errores de pulsación.

- **Container Queries**

— **@container** : Esta es la regla de arroba (at-rule) que permite aplicar estilos a un elemento basándose en las dimensiones o características de su contenedor padre más cercano que haya sido definido como un "contenedor de consulta", a diferencia de las Media Queries, que responden al tamaño total de la ventana gráfica (viewport), el motor de CSS evalúa aquí el espacio físico real donde vive el componente.

Se utiliza como la herramienta definitiva para la modularidad extrema; permite que un mismo componente (como una tarjeta de producto) cambie su diseño automáticamente dependiendo de si está ubicado en una barra lateral estrecha o en el área de contenido principal ancha, garantizando que el diseño sea verdaderamente independiente del contexto de la página.

— **container-type** : Directiva técnica que transforma a un elemento en un contexto de consulta de contenedor, permitiendo que sus descendientes puedan medir sus dimensiones para aplicar reglas condicionales, sus valores principales son size (para consultas de ancho y alto) y inline-size (el más común, enfocado solo en el ancho del flujo de texto).

Se utiliza para declarar nodos estratégicos en el DOM que servirán como puntos de referencia; sin esta propiedad, el navegador no registraría el elemento como un contenedor válido para ser consultado por la regla @container, siendo el interruptor necesario para habilitar la lógica de diseño basada en el espacio disponible.

— **container-name** : Asigna un identificador único o nombre a un contenedor específico, facilitando que las reglas de @container apunten a un ancestro concreto en lugar de simplemente al más cercano, el motor de renderizado utiliza este nombre para establecer una relación explícita entre la consulta y el contenedor deseado.

Se emplea de forma estratégica en estructuras de componentes anidados, por ejemplo permite que un elemento interno consulte específicamente el tamaño de la sección principal de la página (main) ignorando los contenedores intermedios (como wrappers o grids), ofreciendo un control granular y preciso sobre qué caja dicta el estilo del componente.

— **container units (cqi, cqb, cqmin, cqmax)** : Medidas de longitud relativas a las dimensiones del contenedor de consulta definido, funcionan de forma análoga a las unidades de viewport, donde 1cqi representa el 1% del tamaño en línea (ancho) del contenedor y 1cqb el 1% del tamaño de bloque (alto).

Se utilizan para lograr una proporcionalidad interna perfecta, permitiendo que el tamaño de la fuente, el padding o los márgenes de un elemento escalen dinámicamente según el tamaño de su propia caja padre, asegurando que el

componente mantenga su armonía visual interna sin importar en qué parte del layout sea inyectado.

26. Transformaciones

- **transform** : Modifica la forma, posición y orientación de un elemento en el espacio bidimensional o tridimensional sin alterar el flujo normal del documento, técnicamente el motor de renderizado aplica una matriz de transformación geométrica al elemento después de que se ha calculado el layout, lo que significa que el elemento puede desplazarse, rotarse o escalarse visualmente sin que sus vecinos se muevan para ocupar su lugar original.

Se utiliza estratégicamente para crear efectos visuales de alta performance, como animaciones de menús, estados de interacción en botones o correcciones de centrado absoluto, permitiendo manipular la estética del nodo con una eficiencia superior al procesamiento de márgenes o posiciones tradicionales.

— **translate()** : Desplaza un elemento de su posición original en los ejes X (horizontal) e Y (vertical) mediante valores de longitud o porcentajes, a diferencia de las propiedades de posición tradicionales, el uso de translate es procesado por la GPU (unidad de procesamiento gráfico), lo que permite un movimiento fluido y de alto rendimiento sin recalcular el layout de la página. Se utiliza de forma estándar para el centrado absoluto perfecto de elementos (en combinación con top: 50% y left: 50%) y para animaciones de desplazamiento, ya que el espacio original que ocupaba el elemento en el flujo del documento permanece intacto, evitando saltos visuales en los elementos vecinos.

— **scale()** : Modifica el tamaño de un elemento aumentando o disminuyendo sus dimensiones originales mediante un factor multiplicador (donde 1 es el tamaño real), el navegador escala tanto el contenido como las proporciones visuales del nodo de forma proporcional o independiente en cada eje. Se emplea estratégicamente en interfaces interactivas para resaltar componentes, como agrandar sutilmente una tarjeta de producto o un botón al pasar el cursor (:hover), permitiendo una jerarquía visual dinámica que capta la atención del usuario sin alterar el tamaño físico que el elemento ocupa en la estructura de la rejilla o el flujo del texto.

— **rotate()** : Permite girar un elemento alrededor de un punto fijo (por defecto su centro) utilizando unidades de ángulo como grados (deg), radianes (rad) o vueltas (turn), el motor de renderizado aplica una rotación bidimensional que altera la orientación visual del componente. Se utiliza para crear efectos de inclinación y dinamismo en el diseño, es la herramienta fundamental para animar iconos de carga (spinners), orientar flechas en menús desplegables o crear composiciones artísticas donde los elementos se presentan de forma asimétrica, aportando una sensación de profundidad y fluidez al diseño de la interfaz.

— **skew()** : Distorsiona un elemento aplicando una inclinación o sesgo a lo largo de los ejes X e Y, transformando sus ángulos rectos en ángulos oblicuos, el

resultado técnico es una deformación paralelográfica que estira la caja del elemento en una dirección específica basándose en grados.

Se utiliza para fines puramente estéticos y de diseño gráfico avanzado; se emplea para crear fondos con cortes diagonales, efectos de velocidad en tipografías o elementos decorativos que rompen la rigidez del modelo de caja tradicional, permitiendo una estética más agresiva y moderna sin necesidad de recurrir a imágenes externas o SVG complejos.

- **transform-origin** : Establece el punto de anclaje o centro de coordenadas sobre el cual se aplican las transformaciones de un elemento, como la rotación, el escalado o el sesgo, por defecto, el motor de renderizado sitúa este punto exactamente en el centro de la caja (50% 50%), pero mediante esta directiva se puede desplazar a cualquier esquina, borde o coordenada específica (píxeles o porcentajes) dentro o fuera del elemento.

Se utiliza de forma crítica en el diseño de animaciones mecánicas y efectos visuales complejos, por ejemplo permite que una aguja de reloj rote desde su base en lugar de su centro, o que una tarjeta se expanda desde una esquina específica, proporcionando un control geométrico total sobre el comportamiento cinético de los componentes en la interfaz.

27. Transformaciones en 3D

- **perspective** : Define la distancia entre el plano del usuario y el plano $Z=0$, determinando la intensidad del efecto de perspectiva aplicado a los elementos hijos transformados en 3D, cuanto menor sea el valor (en píxeles), más drástica y deformada será la distorsión visual, un valor mayor suaviza la inclinación, emulando la visión desde una mayor distancia.
Se utiliza como la propiedad maestra para habilitar la profundidad en una escena, sin ella las rotaciones en los ejes X o Y se perciben como un simple aplastamiento bidimensional, siendo esencial para crear componentes como cubos giratorios o galerías con efecto de túnel.
- **perspective-origin** : Define el punto de fuga o la posición de la cámara desde la cual el usuario observa los elementos transformados en 3D dentro del contenedor de perspectiva, por defecto, el motor de renderizado sitúa la cámara en el centro (50% 50%), pero mediante esta directiva se puede desplazar hacia cualquier coordenada, alterando el ángulo de visión de la profundidad.
Se utiliza para ajustar la percepción del observador, por ejemplo permite que una cuadrícula de elementos parezca que se aleja hacia una esquina específica de la pantalla o que el usuario está mirando la escena desde "arriba" o desde un lateral, aportando dinamismo a la composición.
- **transform-style** : Determina si los elementos hijos de un contenedor se renderizan en un plano compartido (plano 2D) o si conservan su propia posición independiente en el espacio 3D, su valor más potente, preserve-3d, indica al navegador que no debe "aplanar" la jerarquía de nodos, permitiendo que los hijos se muevan detrás o delante de otros elementos en el eje Z.

Se emplea de forma crítica en la construcción de objetos 3D complejos, como dados, libros que se abren o carruseles cilíndricos, donde es necesario que las diferentes caras del objeto mantengan su relación espacial tridimensional entre sí.

- **backface-visibility** : Controla si la cara posterior (reverso) de un elemento debe ser visible o no cuando este se gira hacia el lado opuesto del espectador mediante una transformación 3D, el motor de renderizado utiliza el valor `hidden` para dejar de dibujar el elemento cuando su ángulo de rotación lo sitúa "de espaldas" a la cámara. Se utiliza de forma estándar en el efecto de "tarjeta que gira" (card flip), permitiendo ocultar el contenido de la cara frontal cuando la tarjeta muestra su reverso, optimizando el rendimiento y evitando que las texturas o textos invertidos se trasluzcan o generen ruido visual innecesario en la animación.
- **translateZ()** : Desplaza un elemento a lo largo del eje Z (hacia adelante o hacia atrás respecto al espectador) en un entorno tridimensional, para que su efecto sea perceptible, el contenedor padre debe tener definida una propiedad de perspectiva, un valor positivo acerca el elemento al usuario (haciéndolo parecer más grande), mientras que un valor negativo lo aleja hacia el fondo de la pantalla. Se utiliza de forma estratégica para crear efectos de profundidad y capas en interfaces complejas, permitiendo que elementos como ventanas flotantes o menús de navegación adquieran un relieve físico real sobre el plano del contenido principal.
- **rotateX()** : Aplica una rotación a un elemento alrededor de su eje horizontal (eje X), provocando un efecto de inclinación hacia adelante o hacia atrás, el motor de renderizado gira la caja del elemento utilizando unidades de ángulo como grados o vueltas. Se emplea habitualmente en el diseño de efectos de perspectiva y tarjetas 3D, es la herramienta fundamental para simular que un elemento se "tumba" sobre la pantalla o para crear animaciones de persiana, donde el contenido parece girar sobre un soporte superior o inferior, aportando un dinamismo volumétrico a la interfaz.
- **rotateY()** : Hace que el elemento gire sobre su eje vertical (eje Y), creando un efecto de puerta que se abre o de una página que se pasa, el navegador recalcula la posición de los puntos de la caja a lo largo del ancho del elemento. Es el componente técnico esencial para el efecto de "card flip" (giro de tarjeta), donde se rota un elemento 180 grados para mostrar su reverso y se utiliza masivamente en galerías interactivas y paneles de información para maximizar el uso del espacio mediante la revelación de contenido oculto tras un giro lateral.
- **rotate3d()** : Función avanzada que permite rotar un elemento alrededor de un eje arbitrario en el espacio tridimensional, definido por un vector $[x, y, z]$ y un ángulo de rotación, en lugar de limitarse a los ejes fijos de la pantalla, el motor de CSS calcula una rotación compleja que puede combinar inclinaciones laterales, verticales y de profundidad simultáneamente. Se utiliza en el desarrollo de objetos 3D personalizados y animaciones orgánicas donde el movimiento no sigue trayectorias ortogonales, permitiendo una libertad geométrica total para orientar componentes en cualquier dirección del espacio.

- **matrix()** : Combina todas las transformaciones 2D (traslación, rotación, escalado y sesgo) en una sola matriz matemática de seis valores [a, b, c, d, e, f], técnicamente es la representación más pura y de bajo nivel de una transformación que el motor de renderizado procesa directamente mediante álgebra lineal.
Aunque es menos legible para los humanos que las funciones individuales, se utiliza habitualmente por herramientas de diseño y librerías de animación para aplicar transformaciones complejas y combinadas de forma ultraeficiente, garantizando una precisión matemática absoluta en el renderizado final del elemento.
- **matrix3d()** : Versión tridimensional de la función de matriz, que utiliza una matriz de 4x4 compuesta por 16 valores para definir cualquier transformación posible en el espacio 3D, esta función controla simultáneamente la posición, rotación, escala, sesgo y perspectiva en los tres ejes de coordenadas.
Se emplea en el desarrollo de motores gráficos web y visualizaciones 3D de alto nivel donde se requiere un control total sobre la proyección geométrica, siendo el método más potente y complejo de CSS para manipular un elemento, permitiendo crear distorsiones de cámara y movimientos espaciales que serían imposibles de lograr con funciones simples.

28. transiciones y animaciones

- **transition** : Propiedad abreviada (shorthand) que permite definir en una sola declaración todos los parámetros de una transición: la propiedad a animar, la duración, la curva de aprendizaje y el retardo, el motor de renderizado procesa esta línea para establecer cómo debe comportarse el cambio de estilo cuando un elemento detecta una modificación en sus propiedades (como al pasar el cursor). Se utiliza para optimizar la escritura del código y garantizar que todos los aspectos de la animación de estado estén sincronizados, permitiendo crear efectos de suavizado profesional en colores, tamaños o posiciones de forma compacta y legible en una sola instrucción.

|
| — **transition-property** : Especifica exactamente qué atributos de CSS deben ser animados durante una transición de estado, el navegador utiliza esta lista de nombres (como background-color, opacity o transform) para monitorizar cambios y aplicar el efecto de interpolación solo a ellos, ignorando el resto.
| Se utiliza para optimizar el rendimiento del navegador, al declarar explícitamente qué propiedades se animan en lugar de usar el valor universal all, se reduce la carga de procesamiento del motor de renderizado, evitando cálculos innecesarios en propiedades que no requieren una transición fluida.

|
| — **transition-duration** : Define el intervalo de tiempo (expresado en segundos o milisegundos) que debe tardar una propiedad en completar su cambio desde el valor inicial al valor final, el motor de CSS utiliza este parámetro para calcular la velocidad de la interpolación de los frames intermedios.
| Es la herramienta fundamental para controlar el ritmo de la interfaz, una duración corta (ejemplo 0.2s) aporta una sensación de respuesta instantánea y ágil, mientras que una duración más larga (ejemplo 1s) crea efectos dramáticos o atmosféricos, permitiendo ajustar la "personalidad" del movimiento en la experiencia de usuario.

— **transition-delay** : Establece un tiempo de espera (en segundos o milisegundos) que debe transcurrir desde que se detecta el cambio de estado hasta que la animación comienza efectivamente a ejecutarse, el navegador pausa la ejecución del cronómetro de la transición basándose en este valor. Se emplea de forma estratégica para crear secuencias de animación escalonadas, permitiendo que diferentes elementos de una lista aparezcan uno tras otro o que un menú secundario espere unos instantes antes de desplegarse, evitando que la interfaz se sienta caótica cuando ocurren múltiples cambios de estado simultáneos.

— **transition-timing-function** : Determina la curva de aceleración (o función de tiempo) que sigue la transición durante su ejecución, el motor de renderizado utiliza fórmulas matemáticas (como ease, linear, cubic-bezier) para dictar si el movimiento comienza lento y acelera, si mantiene una velocidad constante o si tiene un efecto de rebote.

Se utiliza para dotar de realismo y fluidez orgánica a las animaciones, permitiendo que los cambios de estilo no se sientan robóticos, emulando leyes físicas de inercia y fricción que hacen que la interacción con los elementos de la pantalla sea mucho más natural y agradable para el usuario.

- **animaciones**

— **@keyframes** : Regla de arroba (at-rule) fundamental que permite definir una secuencia de estados intermedios para una animación de CSS, estableciendo los fotogramas clave (keyframes) que el navegador debe renderizar, a diferencia de las transiciones, que solo tienen un punto de inicio y uno de fin, el motor de renderizado utiliza esta regla para procesar una lista de porcentajes (del 0% al 100%) o palabras clave (from y to), donde el desarrollador especifica cambios en las propiedades en momentos exactos de la línea de tiempo.

Se utiliza para crear animaciones complejas y continuas que no dependen necesariamente de una interacción del usuario, permitiendo ciclos infinitos, movimientos coreografiados de múltiples pasos y transformaciones dinámicas que dotan de vida y personalidad a la interfaz sin necesidad de scripts externos.

— **animation-name** : Vincula un elemento de la interfaz con una regla de arroba @keyframes previamente definida, asignándole el identificador específico que el motor de renderizado debe buscar, técnicamente funciona como el selector de la secuencia de fotogramas, indicando al navegador qué lista de pasos y cambios de estado debe procesar para ese nodo en particular.

Se utiliza para declarar la identidad de la animación, permitiendo reutilizar una misma lógica de movimiento (como un parpadeo o una rotación) en múltiples elementos del documento simplemente llamando al nombre asignado, facilitando la modularidad y la coherencia visual de los efectos dinámicos en todo el sitio.

- | — **animation-duration** : Define el intervalo de tiempo total que debe transcurrir para que una animación complete un ciclo único desde el primer fotograma (0%) hasta el último (100%), el motor de CSS utiliza este valor (expresado en segundos o milisegundos) para determinar la velocidad a la que se deben interpolar los cambios de las propiedades declaradas.
| Es la herramienta fundamental para establecer el tempo de la narrativa visual, una duración breve genera una sensación de energía y rapidez, mientras que una duración extensa permite apreciar detalles sutiles o crear efectos atmosféricos de fondo, garantizando que el movimiento se ajuste a la intención comunicativa del diseño.
- | — **animation-delay** : Establece un periodo de espera antes de que la animación comience su ejecución efectiva una vez que el elemento ha sido cargado o la propiedad ha sido aplicada, el navegador mantiene el elemento en su estado inicial (o el estado definido por el primer fotograma si se usa un modo de relleno específico) durante el tiempo indicado.
| Se emplea de forma estratégica para coreografiar secuencias complejas, permitiendo que diferentes componentes de la interfaz entren en escena de forma escalonada o rítmica, evitando que todas las animaciones ocurran simultáneamente y permitiendo guiar la atención del usuario de manera ordenada a través de la página.
- | — **animation-timing-function** : Determina la distribución del tiempo dentro del ciclo de la animación, dictando cómo se acelera o desacelera el movimiento entre los fotogramas clave, el motor de renderizado aplica funciones matemáticas (como ease-in, linear o steps) para calcular la progresión del cambio, por ejemplo la función steps() permite crear animaciones "por cuadros" similares al stop-motion o al movimiento de las manecillas de un reloj.
| Se utiliza para dotar de realismo físico y carácter al movimiento, permitiendo que las animaciones no sean lineales y aburridas, sino que sigan curvas de velocidad que emulen la inercia o la gravedad.
- | — **animation-composition** : Determina cómo debe comportarse el motor de renderizado cuando múltiples animaciones afectan simultáneamente a la misma propiedad (especialmente en transformaciones o filtros), sus valores permiten decidir si los valores de las diferentes animaciones deben reemplazarse (replace), sumarse (add) o combinarse proporcionalmente (accumulate).
| Se utiliza para gestionar capas de movimiento complejas, por ejemplo permite que un elemento tenga una animación base de balanceo y que se le "sume" una animación de vibración temporal sin que una anule a la otra, ofreciendo un control matemático superior sobre la mezcla de efectos dinámicos.